

Linux 离线安装包 (flocks-offline.run)

给不能联网的 CentOS Stream 9 / RHEL 9 系机器用的单文件安装包。客户只需要 `scp` 上去、`sudo bash /var/tmp/flocks-offline.run`，装完浏览器开 `http://服务器IP:5173`。包里带齐 Python 3.12、Node.js、uv、预建好的 `.venv`、构建好的 WebUI，可选再带一份 Pro bundle；安装过程和之后的 Pro 升级都不需要 PyPI、npm、GitHub。

这套东西和 `install.sh` / Docker / Windows 包互不影响：装到机器上的形态还是"仓库根 + `.venv` + `webui/dist`"，只是放在 `/opt/flocks/flocks`、以 `flocks` 用户跑、由 `/etc/flocks/flocks.env` 提供环境变量。代码里为它加的逻辑全部由 `FLOCKS_PRO_BUNDLE_DIR`、`FLOCKS_UPDATE_CHANNEL` 和 bundle manifest 的 `prebuilt:true` 触发，这三样只有这个安装包会设置。

内容

- [文件](#)
- [怎么构建](#)
- [装到机器上是什么样](#)
- [Pro 升级在离线机器上怎么走](#)
- [和原来的升级方式是什么关系](#)
- [测试](#)

文件

文件	作用
<code>build-in-docker.sh</code>	一键打包：起 CentOS Stream 9 容器跑下面的构建脚本， <code>--arch x86_64</code>
<code>export-source.sh</code>	从 git 导出干净源码树（ <code>git archive HEAD</code> + 锚定在仓库根的 <code>exclude pathspec</code> ，去掉 <code>tests / packaging / docs</code> 等顶层开发目录），核对导出文件数等于 git 列出的数，再对结果做敏感文件复检； <code>build-offline-run.sh</code> 与 <code>build-in-docker.sh</code> 都靠它拿源码
<code>build-offline-run.sh</code>	真正的构建脚本，在目标发行版的容器里跑（推荐 <code>quay.io/centos/centos:stream9</code> ），产出

文件	作用
	<code>flocks-offline.run</code> + <code>.sha256</code> + <code>versions.json</code>
<code>versions.manifest.json</code>	锁定的 Python / Node / uv / makeself 版本与下载地址模板；升级工具链改这里
<code>installer/install.sh</code>	打进包里的安装脚本， <code>sudo bash xxx.run</code> 时由 makeself 自动执行，全程不提问
<code>installer/flocks.env.template</code>	渲染成 <code>/etc/flocks/flocks.env</code>
<code>installer/flocks.service</code>	渲染成 <code>/etc/systemd/system/flocks.service</code>
<code>installer/flocks-cli-wrapper.sh</code>	渲染成 <code>/usr/local/bin/flocks</code> ， <code>sudo flocks status</code> 会以服务用户身份执行
<code>smoke-test.sh</code>	在无网络的一次性容器里装一遍：装在非默认端口（默认 5273， <code>SMOKE_PORT</code> 可改）、建管理员、本地 Pro 包安装、离线模式拒绝联网升级、不带变量重跑安装包要沿用端口、不带 Pro 的新包覆盖已激活实例（沿用 / 拒绝两种，拒绝那次核对没有新的 <code>.bak</code> 目录且解包目录被挪到 <code>.failed-*</code> ）、 <code>SMOKE_INIT=systemd</code> 时重启容器验 Pro 存活与真 <code>firewalld</code> 永久规则、降级到 OSS 的 <code>prebuilt</code> 重启（放在重启之后，因为重启那段要求 Pro 还在）、 <code>FLOCKS_OFFLINE_SKIP_START=1</code> 只装不启动，输出 PASS/FAIL
<code>verify-install.sh</code>	在装好的真机上跑的验收脚本（root）：布局、 <code>systemd</code> 、 <code>firewalld</code> 、建管理员/登录、本地 Pro 包安装、 <code>--rerun</code> 重复执行安装包，输出 PASS/FAIL。 <code>--port</code> 默认读机器上的 <code>FLOCKS_PORT</code> （ <code>--rerun</code> 会把它作为显式端口传给安装脚本，写死 5173 会把 5273 的实例悄悄挪走）；不传 <code>--admin-pass</code> 时用包内 Python 的 <code>secrets</code> 生成随机密码（原来的 <code>tr </dev/urandom head -c 16</code> 在 <code>pipefail</code> 下必定 exit 141）

CI: `.github/workflows/linux-offline-run.yml` 在 `x86_64 (ubuntu-latest)` 和 `aarch64 (ubuntu-24.04-arm)` 两个原生 runner 上各起一个 CentOS Stream 9 容器构建, 手动触发上传两个 artifact (可填 Pro wheel 的 URL 和 release id), 发布 Release 时把 `flocks-offline-x86_64.run`、`flocks-offline-aarch64.run` 挂到 Release 资产; 交付给客户时改名成说明书写死的 `flocks-offline.run`。

怎么构建

一键 (本机有 docker 即可) :

```
packaging/linux/build-in-docker.sh --arch all --flockspro-wheel
    ../flockspro/dist/flockspro-<ver>-py3-none-any.whl
# 产物: packaging/linux/Output/x86_64/flocks-offline.run、
      packaging/linux/Output/aarch64/flocks-offline.run
```

`--arch` 默认是本机架构; 另一种架构要 Docker 里有 QEMU (`docker run --privileged -rm tonistiigi/binfmt --install all`), 慢好几倍, 同架构机器或 CI 矩阵更快。国内机器加 `--cn` 走 `npmmirror`。`--keep-container` 会留下容器, 进去 `build-offline-run.sh -repack` 可以只换脚本快速重打。

源码取自 `git archive HEAD`, 不是工作区: 有未提交改动时脚本直接拒绝 (`versions.json` 里的 `git_sha` 要能对上真实提交), 本地调试可以加 `--allow-dirty`, 这时会把工作区里已跟踪 + 未忽略的文件一起导出, `versions.json` 记 `source_dirty: true`。`build-in-docker.sh` 在本机导出到 `~/.cache/flocks-offline-build/<arch>/source/` 再只读挂进容器 (上一次的导出改名为 `source.previous-<时间戳>` 留着, 攒多了自己清), 容器里没有工作区可 `tar`。排除用的是 `git pathspec (: (exclude) tests` 这种, 只认仓库根那一层), 不是 `tar --exclude: bsdtar` 的 `exclude` 不锚定, `./assets` 会把 hub 技能里嵌套的 `assets/` 一并吃掉——第一次在 macOS 上导出就丢了 279 个文件, 包装上能跑, 谁也没发现, 所以现在导出后还会核对文件数 (导出数 $\neq$ git 列出的数直接失败)。无论哪种模式, 导出后都会复检一遍: `.env`、`.secret.json`、`*.pem` / `*.key`、`id_rsa*`、`*.log`、`.flocks/{config,data,run,logs,workspace}` 和本地的 `flocks.json` / `mcp_list.json` 只要出现就构建失败 (`*.example` 除外)。之前是 `tar` 整个工作区, 会把 `.flocks/.storage_migrated` 这类本地状态、甚至 `.env` 一起打进客户的包里。

`venv` 不能搬家 (`pyenv.cfg`、`shebang`、`editable .pth` 全是绝对路径), 所以构建必须在目标路径 `/opt/flocks` 下进行, 要求这个目录为空——这就是一定要在干净容器里跑的原因。手动等价命令:

```
docker run --rm -it -v "$PWD":/src -v ~/.cache/flocks-offline-
    build:/root/.cache/flocks-offline-build \
    quay.io/centos/centos:stream9 bash -c '
```

```
dnf -y install tar gzip findutils which git python3 diffutils procs-ng
iproute &&
cd /src && bash packaging/linux/build-offline-run.sh --output-dir
/root/.cache/flocks-offline-build/out'
```

带 Pro bundle: 加 `--flockspro-wheel /path/flockspro-<ver>-py3-none-any.whl`, 可选 `--pro-release-id` (console 上对应的 release)、`--pro-bundle-version`。wheel 必须是和这个 core 配对的版本。

国内构建机可以走镜像:

```
FLOCKS_NODE_BASE_URL=https://registry.npmmirror.com/-/binary/node、
FLOCKS_NPM_REGISTRY=https://registry.npmmirror.com/、
FLOCKS_UV_DEFAULT_INDEX=https://mirrors.aliyun.com/pypi/simple。下载的工具链缓
存在 ~/.cache/flocks-offline-build, 改 manifest 才会重新下。
```

构建做的事: 下载工具链 → `export-source.sh` 导出源码 (git archive, 不含 .git、tests、packaging、docs、node_modules) 并复检 → 建完包 python3
`~/claude/tools/artifact_scan.py Output/<arch>/flocks-offline.run --repo`。再看一眼包里面有没有不该有的东西 → 用包内 Python 跑 `uv sync --frozen --group dev` → 在 scratch 目录 `npm ci && npm run build` 后只把 dist 拷进来 → 写 Pro bundle 和 `versions.json` → `makeself` 打包 (`--target /opt/flocks/.staging`, 解包不占 /tmp)。x86_64 和 aarch64 都能构建, 产物只能装到同架构机器。

装到机器上是什么样

<code>/opt/flocks/tools/{python,node,uv}</code>	运行时, root 所有
<code>/opt/flocks/flocks</code>	仓库副本 + .venv + webui/dist, flocks 用户所有 (升级要写它)
<code>/opt/flocks/bundle</code>	Pro bundle (manifest.json + wheels/), 有才装
<code>/opt/flocks/cache/uv</code>	构建机的 uv 缓存, UV_OFFLINE=1 时兜底
<code>/opt/flocks/installer, versions.json</code>	这次安装用的脚本和版本信息
<code>/etc/flocks/flocks.env</code>	服务环境变量; 改 IP/端口/控制台地址就改它然后 <code>systemctl restart flocks</code>
<code>/etc/systemd/system/flocks.service</code>	oneshot + RemainAfterExit, <code>ExecStart=flocks start, ExecStop=flocks stop</code>
<code>/var/lib/flocks/.flocks</code>	数据 (配置、数据库、日志、授权), 任何情况下不动
<code>/usr/local/bin/flocks</code>	CLI 包装

安装脚本会：先做预检（架构、磁盘、端口、已激活 Pro 与 bundle 的匹配），预检不过就报错退出、不动任何文件，并把已解包的安装文件挪到 `<解包目录>.failed-<时间戳>`（makeself 失败时不清 `--target` 目录，不挪开的话下一个包会解在它上面、两个版本的文件混在一起）；建 `flocks` 用户；已有安装先停服、把旧的 `flocks/`、`tools/`、`bundle/` 改名成 `*.bak-<时间戳>`（不删）；搬入新文件后 `restorecon -R`（从临时目录 `mv` 过来的文件会带 `tmp_t` 标签，SELinux enforcing 下 `systemd` 起不来）；合并 `flocks.env`（受管的几个键覆盖，其它保留）；如果之前激活过 Pro（`~/flocks/run/pro-bundle-installed.json` 在），把包里的 Pro wheel 重新装进新 `venv`；新包不带 `bundle/` 时，先看机器上原来的 `/opt/flocks/bundle` 能不能沿用（`manifest.json` 的 `core_version` 等于新包 `core`、`wheel sha256` 对得上 `manifest`），能就原样搬回来继续装 Pro，不能就在动任何文件之前报错退出（“这台机器已激活 Flocks Pro，但新安装包不含 Pro 组件.....本次未做任何改动”），不会把一台 Pro 实例静默装成 OSS；`systemctl enable --now`；`firewalld` 放行服务端口（默认 `zone` 加上所有绑了网卡/来源的 `zone`，永久规则和运行态规则分别核对补齐——之前只查运行态，运维手动 `--add-port` 过一次的机器永久规则就永远不会写，`reload` 或重启后端口关闭；`firewalld` 拒绝写入只打警告、给出手动命令，不会让已经换好文件的安装中途退出）；`chronyd` 打开；轮询 `/api/health` 直到 200；最后打印 `[flocks] 安装完成: http://<IP>:<端口>`。日志在 `/var/log/flocks-offline-install.log`。

端口只有一个来源：本次显式给的 `FLOCKS_OFFLINE_PORT` > 机器上

`/etc/flocks/flocks.env` 里已有的 `FLOCKS_PORT` > 5173，`env` 文件、`firewalld`、健康检查和最后那行 URL 用的都是它。所以装在 5273 上的实例，之后直接 `sudo bash 新包.run` 不带变量也还是 5273（之前会退回 5173 去查健康，等 180 秒后报“没有就绪”）；要换端口就显式传 `FLOCKS_OFFLINE_PORT`，这时 `env` 里的 `FLOCKS_PORT` 才会被改写（`env` 里那行本身是坏值时也会被改写成 5173，否则服务起不来）。装好后留在 `/opt/flocks/installer/` 的脚本副本只供查看，直接执行会被拒绝——它的解包目录就是安装目录，跑起来会把自己所在的安装挪成 `.bak`。

`systemd` 用 `oneshot` 而不是把 `service-daemon` 放前台，是因为在线升级的

`restart_handoff` 会先 `stop_all` 再自己拉起新的 `flocks start`；前台模式下主进程一退 `systemd` 会连 `handoff` 一起杀掉。`oneshot` 下 `systemctl status` 只能看到 `active (exited)`，看进程用 `sudo flocks status`。

调试用的环境变量（客户不需要）：`FLOCKS_OFFLINE_DRY_RUN=1` 只打印计划；

`FLOCKS_OFFLINE_SKIP_SYSTEMD=1` 在容器里用 `flocks start` 直接起；

`FLOCKS_OFFLINE_SKIP_FIREWALL=1`、`FLOCKS_OFFLINE_SKIP_START=1`（只装不启动，`smoke` 里有真跑；之前这条分支调了一个还没定义的函数，`exit 127`）、

`FLOCKS_OFFLINE_PORT`、`FLOCKS_OFFLINE_HEALTH_TIMEOUT`。

Pro 升级在离线机器上怎么走

页面上"开始升级"调的还是 `perform_pro_bundle_install`。它先看 `FLOCKS_PRO_BUNDLE_DIR`（安装包写进 env 的 `/opt/flocks/bundle`）：目录里有 `manifest.json` 就用本地包，不下载；manifest 标了 `prebuilt: true` 且没带 `flocks/` 源码，就只做 `uv pip install --no-deps <wheel>`，跳过 `uv sync` 和 `npm`，写 marker 后走 handoff 的 Pro-only 重启。console 的 manifest 只是尽力拉一次：bundle 版本相同、且双方都带的 `core_version` / `flockspromodule_version` / `build_id` 都一致时，只把 `release_id` / `bundle_release_id` 合进 marker；其余字段一律以本地包为准（同一个 bundle 版本可以配不同 wheel 重打，不能因为版本号相同就把 console 那份的组件版本和 build id 记成已装）；拉不到、版本不同或身份冲突都不合并，也不影响安装。

`FLOCKS_UPDATE_CHANNEL` 被安装包固定成 `flockspromodule-offline-<core 版本>`，心跳和升级申请都会带上这个 `channel` 字段；console 给每个离线包版本开一个冻结渠道、发和包里同一个 release，在线客户的 `flockspromodule` 渠道不受影响。老客户端不带 `channel` 字段，console 按 `flockspromodule` 处理。

页面上“降级到 OSS”走 `perform_pro_bundle_downgrade`：卸掉 `flockspromodule` wheel 后的 handoff 重启在离线实例上（`FLOCKS_DEPLOY_MODE=offline`）带 `--prebuilt`，跳过 `uv sync` / `npm install` / `npm run build` 直接 `flocks start`；在线实例的降级 handoff 不变。

`prebuilt` 格式的 bundle 只能发到离线渠道。老版本客户端拿到没有 `flocks/` 目录的包会判成"不是 Pro bundle"，直接报"未找到 `flockspromodule` wheel"。

和原来的升级方式是什么关系

离线实例的升级机制和在线安装是同一套代码（备份到 `~/.flocks/version`、`_replace_install_dir` 保留 `.venv`、marker、handoff 重启），只是"包从哪来"不同：Pro 组件来自包内 `bundle/`，核心升级靠新版 `.run` 覆盖执行。env 里的 `FLOCKS_DEPLOY_MODE=offline` 会让页面"检查更新 → 升级"、`flocks update` 和 `/api/update/apply` 在动任何文件之前就拒绝联网升级并提示用新 `.run`（否则会先替换源码再在 `uv sync` 上被 `UV_OFFLINE=1` 卡住，服务停在半路）。页面上 `update_allowed=false` 时显示"离线安装包部署"的提示，不是 Docker 那条；这块提示不看 `has_update`。核心版本检查（`edition=flocks`）在离线实例上根本不出网：`check_update` 直接返回本地版本、`latest` 为空、无错误——之前会挨个等 GitHub/Gitee 超时（30—45 秒），最后一级兜底还会在当前目录跑 `git tag`，在别的 git 仓库里执行 CLI 甚至能凑出一个假的“有新版本”。页面这时显示“离线部署，未查询远端版本”而不是“已是最新”；`flocks update` / `--check` 先打

提示、再打当前版本，退出码 0，不问“是否使用中国镜像”。Pro 那条

(`edition=flockspiro`，走 `console manifest`) 不受影响，`console` 是这类机器唯一允许访问的端点。

机器如果后来能联网、想改回在线升级：从 `/etc/flocks/flocks.env` 删掉

`FLOCKS_DEPLOY_MODE` / `FLOCKS_OFFLINE_INSTALL` / `UV_OFFLINE` / `npm_config_offline` / `FLOCKS_PRO_BUNDLE_DIR` / `FLOCKS_UPDATE_CHANNEL`，`systemctl restart flocks`，之后就是普通源码安装的在线升级路径（GitHub/Gitee 源码包 + `console flockspiro` 渠道），代码不用改。反过来，`install.sh` 装的实例不能用 `.run` 原地升级：`.run` 是另一种部署形态（`/opt/flocks`、`flocks` 用户、`/var/lib/flocks/.flocks`），数据要自己从 `~/.flocks` 搬过去，而且 5173 被旧实例占着时安装脚本会直接报错。

在线用户不受影响：新逻辑全部靠 `FLOCKS_PRO_BUNDLE_DIR`、

`FLOCKS_DEPLOY_MODE=offline`、`manifest` 的 `prebuilt` 和 `handoff` 的 `--prebuilt` 参数触发，`service_manager` 在 `ps` 不可用时（没有这个命令，或 `BusyBox` 那种不认 `-p` / `-eo` 的 `ps`，总之调用失败）改读 `/proc`——最小化容器里 `flocks stop` 找不到守护进程会把旧后端留在端口上；`ps` 正常的机器走原来的路径，`console` 现在发的 `bundle`（带 `flocks/`）新代码照收，既有 `updater` 测试全部原样通过。

测试

```
uv run pytest tests/scripts/test_linux_offline_installer.py # 语法、函数先定义后
调用、无递归删除、模板键、dry-run (含端口沿用/覆盖)、源码导出不带本地密钥、已激活
Pro 的沿用/拒绝、firewalld 永久规则 (假 firewall-cmd)、随机密码
uv run pytest tests/updater/test_updater_prebuilt_bundle.py # 本地 bundle /
prebuilt 路径、离线降级 handoff 带 --prebuilt、console 身份只在完全匹配时借
release_id, 以及默认路径不变的回归
uv run pytest tests/cli/test_update_command.py # 离线实例 flocks
update 先给 .run 指引
cd webui && npx vitest run src/components/common/UpdateModal.test.tsx # 离线提
示不依赖 has_update
uv run pytest tests/cli/test_service_manager.py -k proc # 没有 ps
时的 /proc 兜底
packaging/linux/smoke-test.sh Output/flocks-offline.run # 无网络容器里真装
一遍 (需要 docker, 镜像架构要和包一致)
sudo packaging/linux/verify-install.sh --port 5173 --rerun /var/tmp/flocks-
offline.run # 在装好的真机上验收
```

真机上 5173 已被别的服务占用时，安装和验收都可以换端口：`FLOCKS_OFFLINE_PORT=5273`

`sudo bash /var/tmp/flocks-offline.run`、`verify-install.sh --port 5273`。安装脚本发现端口被占且没有旧的 `Flocks` 安装会直接报错退出，不会去停别人的进程。

真机验收清单在 `webook_dev/docs/offline_install/` 的差距分析文档里（断外网只放行 `portal/passport` 的 CentOS Stream 9 x86_64，按客户说明书原文从 `scp` 跑到 `Pro` 激活、两次重启、重复执行 `.run`）。